Exploratory Data Analysis: Home Credit Default Risk

IS 6812 MSBA Capstone 1

AUTHOR
Marco Rodriguez

PUBLISHED
September 21, 2026

1 Introduction

1.1 Business problem

Home Credit lends to customers with little or no formal credit history, making roughly six million loan decisions per month, many in under a minute at the point of sale. Because these applicants have thin or missing credit bureau files, the company cannot rely on the conventional bureau score that anchors underwriting elsewhere, and its decisions carry avoidable error in both directions: approving applicants who cannot repay creates direct credit losses, and declining applicants who would have repaid costs revenue, lifetime value, and progress on the company's mission of expanding credit access. The project goal is a repayment-risk score, built from data the company already holds, that reduces defaults among approved loans while increasing approvals of creditworthy thin-file applicants.

This notebook covers the data understanding phase of CRISP-DM. Its job is to learn the structure of the application data, find its problems, and decide what to do about them before data preparation and modeling begin.

1.2 Questions

The questions below come from three sources: claims in my business problem statement (BPS), a feature report of the risk factors regulators and lenders use (`docs/feature_report.md`), and the standing data integrity checks. Each question is tied to the decision its answer informs.

#	Question	Source	Decision it informs
1	What does a wrong decision cost the lender, and can this data measure it?	BPS: false approvals and false declines have different costs	How the model will be evaluated
2	What share of applicants are thin-file, and do they default differently?	BPS: applicants are thin-file	Whether thin-file applicants need separate treatment
3	How well do the external scores separate defaulters,	Feature report: credit history	Which features anchor the model; how to handle missing scores

#	Question	Source	Decision it informs
	and does combining them help?		
4	Is reported income believable, and is payment-to-income predictive?	Feature report: ability to repay (CFPB)	Whether to build and trust PTI
5	How often does the loan exceed the goods price, and is loan-to-value predictive?	Feature report: collateral value (World Bank)	Whether to build LTV
6	Does the ratio of credit to annuity (implied term) relate to default?	Feature report: loan maturity	Whether to build an implied-term feature
7	Does employment tenure predict default once the placeholder value is handled?	Feature report: employment status (CFPB)	How to encode tenure and pensioners
8	Does recent credit-seeking behavior signal risk?	Feature report: credit inquiries (World Bank)	Whether inquiry counts earn a place in the model
9	Do residential and identity stability indicators relate to default?	Feature report: length at address (World Bank)	Which stability features to keep
10	Which applicant characteristics carry the most signal overall?	Standing checks and data map	Feature shortlist for modeling
11	How strongly do protected attributes relate to default, and which predictors act as proxies for them?	BPS: decisions must be fair and explainable	Which columns to exclude or monitor

2 Description of the data

2.1 How is the data loaded and prepared for analysis?

The notebook reads the two application files and the data dictionary, then defines three helper functions used throughout: a signal measure (univariate AUC), a default-rate table, and a standard bar chart.

```
# Core libraries for data handling, plotting, and a quick signal metric
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import seaborn as sns
from sklearn.metrics import roc_auc_score

# Consistent plot styling across the notebook
sns.set_theme(style="whitegrid", context="notebook")
PRIMARY, ACCENT, GREY = "#1f5f8b", "#c0392b", "#7f8c8d"
pd.options.display.float_format = "{:,.4f}".format

# Paths are relative to the notebooks/ folder of the project repository
DATA = Path("../data/raw")
train = pd.read_csv(DATA / "application_train.csv")
test = pd.read_csv(DATA / "application_test.csv")
# The dictionary ships in Latin-1 encoding, not UTF-8
dictionary = pd.read_csv(DATA / "HomeCredit_columns_description.csv", encoding="latin-
        1")

BASE_RATE = train["TARGET"].mean()  # overall default rate, used as a reference line in
        plots
OUTCOME = {0: "Repaid", 1: "Payment difficulty"}           # readable labels for the
        target
OUTCOME_COLORS = {"Repaid": PRIMARY, "Payment difficulty": ACCENT}
```

```
# Helper functions reused throughout the notebook

def auc_signal(x, y=train["TARGET"]):
    """Univariate AUC on non-missing rows, folded so 0.5 = no signal and 1.0 = perfect.
    Also returns the direction: '+' means higher values go with more default."""
    mask = x.notna()                                       # AUC cannot use missing
        values
    a = roc_auc_score(y[mask], x[mask])                    # 0.5 = random ranking
    return max(a, 1 - a), ("+" if a > 0.5 else "-")

def rate_by_group(frame, group, target="TARGET"):
    """Default rate and applicant count for each level of a grouping variable."""
    return (frame.groupby(group, observed=True)[target]
                 .agg(default_rate="mean", applicants="size")
                 .reset_index())

def plot_rate(table, x, title, xlabel, rotate=0, ax=None):
```

```
    """Bar chart of default rate by group, with the overall default rate as a dashed
        reference."""
    ax = ax or plt.gca()                                    # draw on the axis passed in,
        or the current one
    sns.barplot(data=table, x=x, y="default_rate", color=PRIMARY, ax=ax)
    # Dashed line at the portfolio default rate, so every bar reads as above or below
        average
    ax.axhline(BASE_RATE, ls="--", color=ACCENT, lw=1.2, label=f"Overall default rate
        ({BASE_RATE:.1%})")
    ax.yaxis.set_major_locator(mtick.MaxNLocator(nbins=6, steps=[1, 2, 5, 10]))  #
        whole-percent ticks only
    ax.yaxis.set_major_formatter(mtick.PercentFormatter(1.0, decimals=0))
    ax.set_title(title, fontweight="bold")
    ax.set_xlabel(xlabel)
    ax.set_ylabel("Default rate")
    ax.tick_params(axis="x", rotation=rotate)
    ax.set_ylim(0, max(table["default_rate"].max(), BASE_RATE) * 1.3)  # headroom so the
        legend never covers a bar
    ax.legend(loc="upper left", frameon=False, fontsize=9)
    return ax
```

2.2 What does one row represent, and how large are the files?

```
# One row per loan application; SK_ID_CURR is the application key
summary = pd.DataFrame({
    "Rows": [len(train), len(test)],
    "Columns": [train.shape[1], test.shape[1]],
    "Key unique": [train["SK_ID_CURR"].is_unique, test["SK_ID_CURR"].is_unique],
    "Has TARGET": ["TARGET" in train, "TARGET" in test],
}, index=["application_train", "application_test"])
summary
```

	Rows	Columns	Key unique	Has TARGET
application_train	307511	122	True	True
application_test	48744	121	True	False

- **Grain:** one row per loan application. Each row describes the applicant and the requested loan at the moment of application.
- **Key:** `SK_ID_CURR`, unique in both files, with no IDs shared between train and test.
- **Target:** `TARGET = 1` marks a client with payment difficulties, and `TARGET = 0` covers all other cases. It exists only in the training file. The dictionary defines difficulty as a late payment of “more than X days on at least one of the first Y installments” and never fills in X or Y, so the exact default threshold is unknown.
- **Supplementary tables:** not used at this stage. The six supplementary tables (bureau, bureau balance, previous applications, POS cash, credit card, installments) join to this table through `SK_ID_CURR` and will be mapped for grain, join path, and coverage in the data preparation stage.

2.3 How are the 122 columns organized?

The 122 columns fall into eleven groups. Grouping them is what makes the dictionary usable: the groups behave differently in missingness, scale, and meaning.

```
# Assign every column to a group using its name pattern
def column_group(col):
    if col in ("SK_ID_CURR", "TARGET"): return "Key and target"
    if col in ("NAME_CONTRACT_TYPE", "AMT_CREDIT", "AMT_ANNUITY", "AMT_GOODS_PRICE"):
        return "Loan terms"
    if col.startswith("EXT_SOURCE"): return "External scores"
    if col.startswith("AMT_REQ_CREDIT_BUREAU"): return "Credit bureau inquiries"
    if col.endswith(("_AVG", "_MODE", "_MEDI")): return "Building and housing
        statistics"
    if "SOCIAL_CIRCLE" in col: return "Social circle defaults"
    if col.startswith("FLAG_DOCUMENT"): return "Documents provided"
    if col.startswith(("FLAG_MOBIL", "FLAG_EMP_PHONE", "FLAG_WORK_PHONE",
        "FLAG_CONT_MOBILE", "FLAG_PHONE", "FLAG_EMAIL")): return "Contact flags"
    if col.startswith(("REGION_", "REG_", "LIVE_")): return "Region and address"
    if col in ("AMT_INCOME_TOTAL", "NAME_INCOME_TYPE", "DAYS_EMPLOYED",
        "OCCUPATION_TYPE", "ORGANIZATION_TYPE"): return "Income and employment"
    return "Applicant profile and application process"

# Map each column to its group, then count columns and average the missing share per
        group
groups = pd.Series({c: column_group(c) for c in train.columns}, name="group")
group_table = (groups.value_counts().rename("Columns").to_frame()
               .join(train.isna().mean().groupby(groups).mean().rename("Avg. share
        missing")))
group_table.style.format({"Avg. share missing": "{:.1%}"})
```

	Columns	Avg. share missing
group		
Building and housing statistics	47	57.9%
Documents provided	20	0.0%
Applicant profile and application process	16	4.2%
Region and address	9	0.0%
Contact flags	6	0.0%
Credit bureau inquiries	6	13.5%
Income and employment	5	6.3%
Loan terms	4	0.0%
Social circle defaults	4	0.3%
External scores	3	25.5%
Key and target	2	0.0%

The groups differ sharply in completeness. The 47 building and housing columns make up more than a third of the file and are mostly empty, while the loan terms, region, contact, and document groups

are essentially complete. The three external scores are only three columns but carry most of the predictive signal (Q3), so their 25% missing rate matters far more than the building block's 58%. Column count is therefore a poor guide to importance, and each group needs its own treatment in data preparation.

2.4 Where does the dictionary disagree with the data?

Every application column appears in the dictionary, and the dictionary has no application entries missing from the data. The definitions, however, do not always match the values.

```
# Check dictionary claims against the data, one discrepancy per row
discrepancies = pd.DataFrame([
    ["DAYS_BIRTH", "Client's age in days", f"All values negative (max
        {train.DAYS_BIRTH.max():,}); counts days before application"],
    ["DAYS_EMPLOYED", "Days before application employment started",
     f"{(train.DAYS_EMPLOYED == 365243).sum():,} rows equal +365,243 (about 1,000 years
        in the future)"],
    ["REGION_RATING_CLIENT_W_CITY", "Rating values 1, 2, 3",
     f"Test file contains {(test.REGION_RATING_CLIENT_W_CITY == -1).sum()} row rated
        -1"],
    ["Building _MODE columns", "Normalized numeric information",
     "Four _MODE columns (FONDKAPREMONT, HOUSETYPE, WALLSMATERIAL, EMERGENCYSTATE) are
        text categories"],
    ["AMT_REQ_CREDIT_BUREAU_YEAR", "Enquiries 'one day year' before application", "Typo
        in the dictionary; window intended as the prior year excluding the last 3
        months"],
    ["AMT_GOODS_PRICE", "Price of goods for consumer loans",
     f"Missing for {train.AMT_GOODS_PRICE.isna().sum()} rows, all of them revolving
        loans"],
    ["TARGET", "Late more than 'X days' on one of the first 'Y installments'", "X and Y
        are never defined, so the default threshold is unknown"],
    ["Currency", "Not stated anywhere", "All AMT_ columns are in an unnamed local
        currency"],
], columns=["Column", "Dictionary says", "Data shows"])
discrepancies.style.hide(axis="index")
```

Column	Dictionary says	Data shows
DAYS_BIRTH	Client's age in days	All values negative (max -7,489); counts days before application
DAYS_EMPLOYED	Days before application employment started	55,374 rows equal +365,243 (about 1,000 years in the future)
REGION_RATING_CLIENT_W_CITY	Rating values 1, 2, 3	Test file contains 1 row rated -1
Building _MODE columns	Normalized numeric information	Four _MODE columns (FONDKAPREMONT, HOUSETYPE, WALLSMATERIAL, EMERGENCYSTATE) are text categories
AMT_REQ_CREDIT_BUREAU_YEAR	Enquiries 'one day year' before application	Typo in the dictionary; window intended as the prior year excluding the last 3 months
AMT_GOODS_PRICE	Price of goods for consumer loans	Missing for 278 rows, all of them revolving loans

Column	Dictionary says	Data shows
TARGET	Late more than 'X days' on one of the first 'Y installments'	X and Y are never defined, so the default threshold is unknown
Currency	Not stated anywhere	All AMT_ columns are in an unnamed local currency

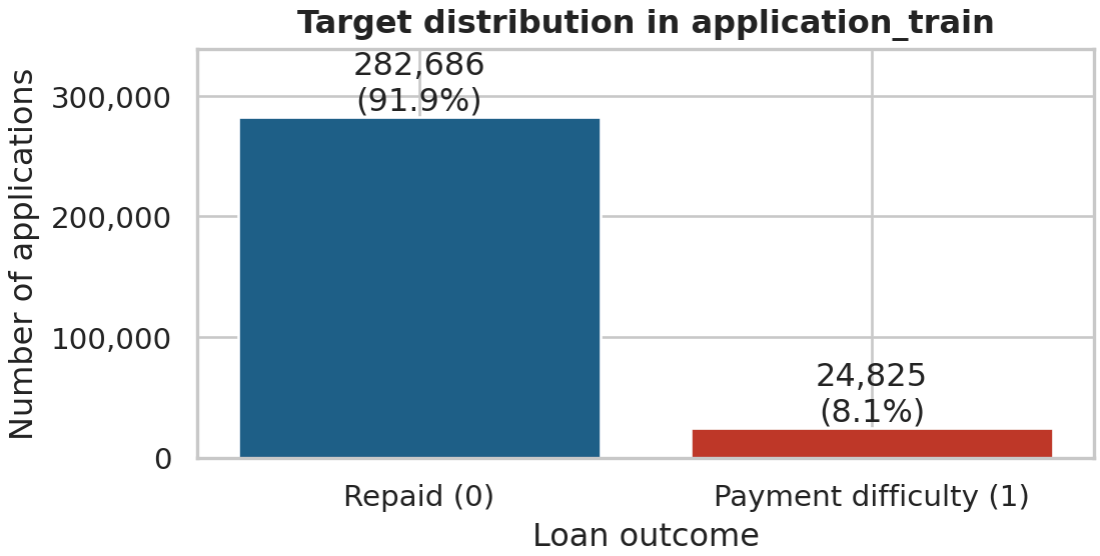
Decision: treat the dictionary as a starting point, not a specification. The `DAYS_*` columns will be converted to positive years, the placeholder and the `-1` rating recoded as missing, and the four text `_MODE` columns handled as categoricals.

3 Data integrity

Each check below follows the same pattern: the test, the finding, and the decision for data preparation.

3.1 How imbalanced is the target?

```
# Share of applications with and without payment difficulties
target_counts = train["TARGET"].value_counts().sort_index()   # index 0 = repaid, 1 =
        difficulty
fig, ax = plt.subplots(figsize=(6, 3.2))
ax.bar(["Repaid (0)", "Payment difficulty (1)"], target_counts.values, color=[PRIMARY,
        ACCENT])
for i, v in enumerate(target_counts.values):                # label each bar with count
        and share
    ax.text(i, v, f"{v:,}\n({v / len(train):.1%})", ha="center", va="bottom")
ax.set_title("Target distribution in application_train", fontweight="bold")
ax.set_xlabel("Loan outcome")
ax.set_ylabel("Number of applications")
ax.set_ylim(0, target_counts.max() * 1.2)
ax.yaxis.set_major_formatter(mtick.StrMethodFormatter("{x:,.0f}"))
plt.tight_layout(); plt.show()
```



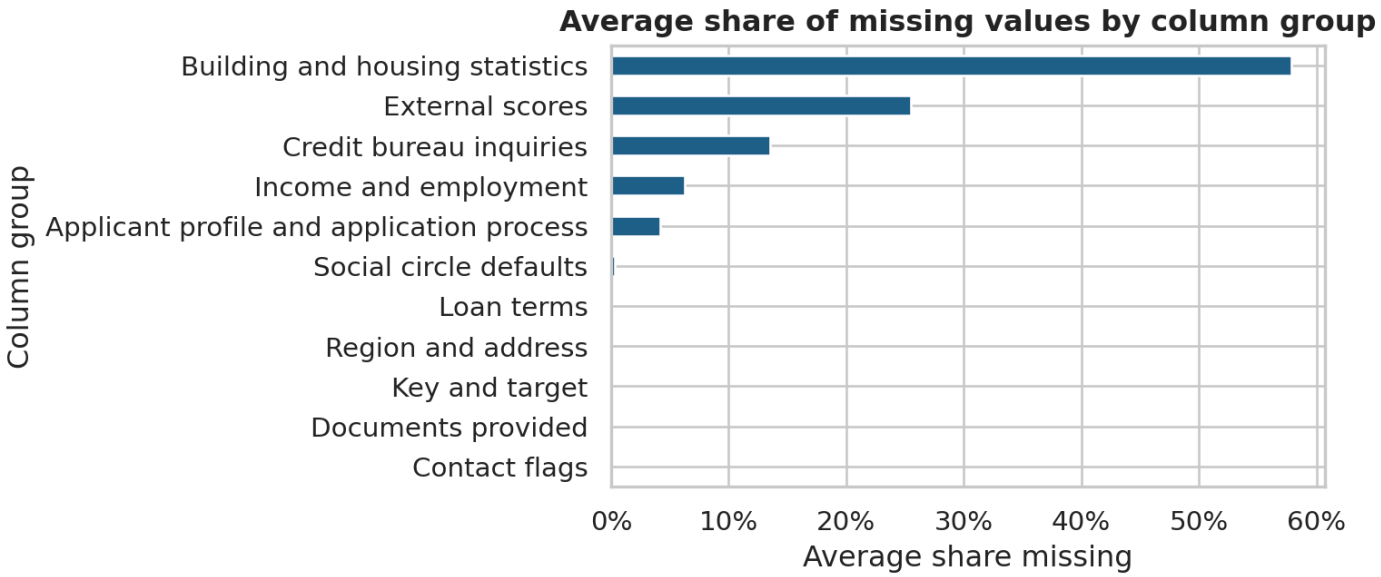
- **Finding:** 24,825 of 307,511 applications (8.07%) ended in payment difficulty. A model that predicts “repaid” for everyone would be 91.9% accurate and completely useless.
- **Decision:** evaluate models on ranking quality (AUC) and on expected cost at a chosen cutoff, never on accuracy. Use stratified splits so every validation fold keeps the 8% rate.

3.2 How much data is missing, and does missingness relate to default?

```
# How much of each column group is missing?
missing_share = train.isna().mean()
cols_missing = missing_share[missing_share > 0]
print(f"{len(cols_missing)} of {train.shape[1]} columns have missing values; "
      f"{(missing_share > 0.4).sum()} are more than 40% missing.")

# Average the missing share within each column group and plot the groups from least to
        most missing
fig, ax = plt.subplots(figsize=(8, 3.6))
(missing_share.groupby(groups).mean().sort_values()
     .plot.barh(ax=ax, color=PRIMARY))
ax.xaxis.set_major_formatter(mtick.PercentFormatter(1.0, decimals=0))
ax.set_title("Average share of missing values by column group", fontweight="bold")
ax.set_xlabel("Average share missing")
ax.set_ylabel("Column group")
plt.tight_layout(); plt.show()
```

```
67 of 122 columns have missing values; 49 are more than 40% missing.
```



Counting missing values is only half the test. The question for a lender is whether *missing* is itself information.

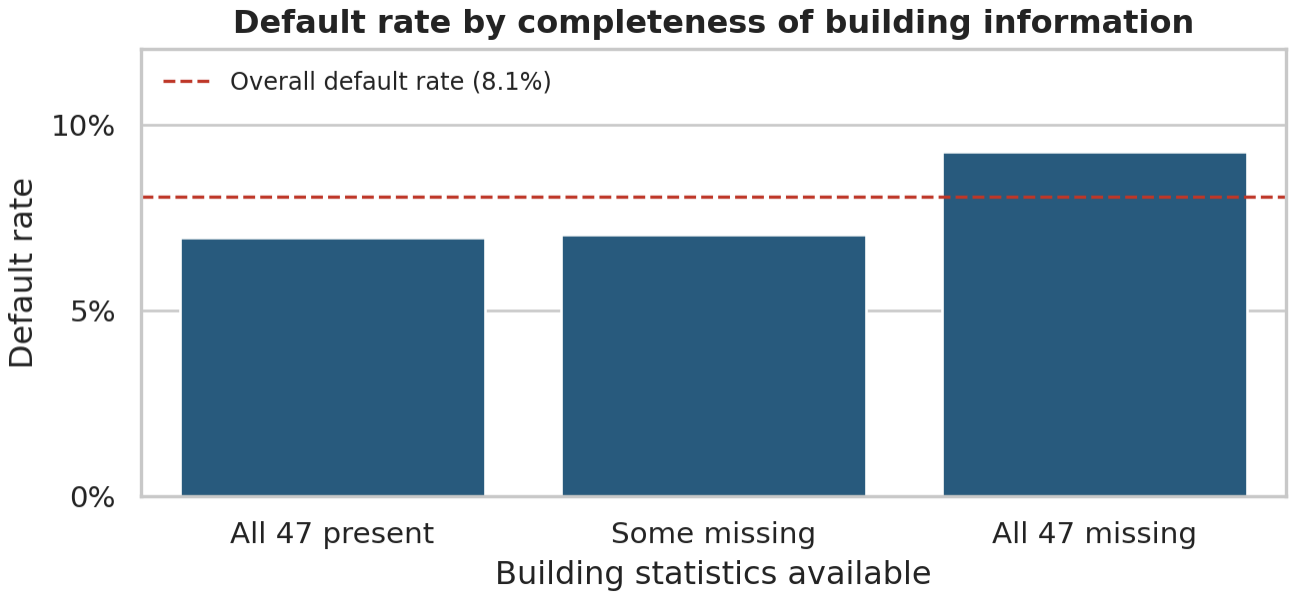
```
# For each column with missing values: default rate when missing vs. when present
rows = []
for col in cols_missing.index:
    miss = train[col].isna()
    if miss.sum() < 100:            # skip columns with too few missing rows to compare
        continue
    rows.append({"column": col, "group": groups[col], "share_missing": miss.mean(),
                 "default_if_missing": train.loc[miss, "TARGET"].mean(),
                 "default_if_present": train.loc[~miss, "TARGET"].mean()})
miss_vs_target = pd.DataFrame(rows)
# Positive gap = applicants with the value missing default more often than those with it
        present
miss_vs_target["gap_pts"] = (miss_vs_target.default_if_missing -
        miss_vs_target.default_if_present) * 100

# Summarize by group so the 47 building columns do not crowd out everything else
by_group = (miss_vs_target.groupby("group")
            .agg(columns=("column", "size"), share_missing=("share_missing", "mean"),
                 default_if_missing=("default_if_missing", "mean"),
                 default_if_present=("default_if_present", "mean"),
                 gap_pts=("gap_pts", "mean"))
            .sort_values("gap_pts", ascending=False))
by_group.style.format({"share_missing": "{:.1%}", "default_if_missing": "{:.2%}",
                       "default_if_present": "{:.2%}", "gap_pts": "{:+.2f}"})
```

	columns	share_missing	default_if_missing	default_if_present	gap_pts
group					
Credit bureau inquiries	6	13.5%	10.34%	7.72%	+2.62

group	columns	share_missing	default_if_missing	default_if_present	gap_pts
Building and housing statistics	47	57.9%	8.93%	6.93%	+2.00
External scores	3	25.5%	8.57%	7.78%	+0.79
Loan terms	1	0.1%	7.55%	8.07%	-0.52
Applicant profile and application process	2	33.2%	6.96%	7.66%	-0.71
Income and employment	1	31.3%	6.51%	8.79%	-2.27
Social circle defaults	4	0.3%	3.53%	8.09%	-4.56

```
# The building block of 47 columns tends to be missing all at once; check that pattern
bcols = [c for c in train.columns if groups[c] == "Building and housing statistics"]
n_missing = train[bcols].isna().sum(axis=1)                 # missing building fields
        per applicant
# Bin applicants into complete, partial, and fully missing building records
block = pd.cut(n_missing, [-1, 0, len(bcols) - 1, len(bcols)],
               labels=["All 47 present", "Some missing", "All 47 missing"])
block_tbl = rate_by_group(train.assign(block=block), "block")
fig, ax = plt.subplots(figsize=(7, 3.4))
plot_rate(block_tbl, "block", "Default rate by completeness of building information",
        "Building statistics available", ax=ax)
plt.tight_layout(); plt.show()
block_tbl.style.format({"default_rate": "{:.2%}", "applicants": "
        {:,}"}).hide(axis="index")
```



block	default_rate	applicants
All 47 present	6.96%	78,595
Some missing	7.04%	83,161

block	default_rate	applicants
All 47 missing	9.26%	145,755

- **Finding:** 67 of 122 columns have gaps, and 49 are more than 40% empty. Missingness is not random. Applicants with no building information default at 9.3%, against 7.0% for those with a complete record; applicants missing credit bureau inquiry data default at 10.3% against 7.7%. The 47 building columns go missing as a block, which points to one mechanism: the lender either has a housing record for the applicant or it does not. Missingness also runs the other way: applicants with no occupation recorded default at only 6.5%, because 57% of them are pensioners, so a blank can mark a safer group as easily as a riskier one.
- **Decision:** do not drop rows. For every column group with material missingness, add a binary "missing" indicator before imputing, so the model can use the absence of data as the signal it is. Collapse the 47 building columns (three statistics of the same 14 measures, plus four categories) to one set of `_AVG` columns and a single completeness flag.

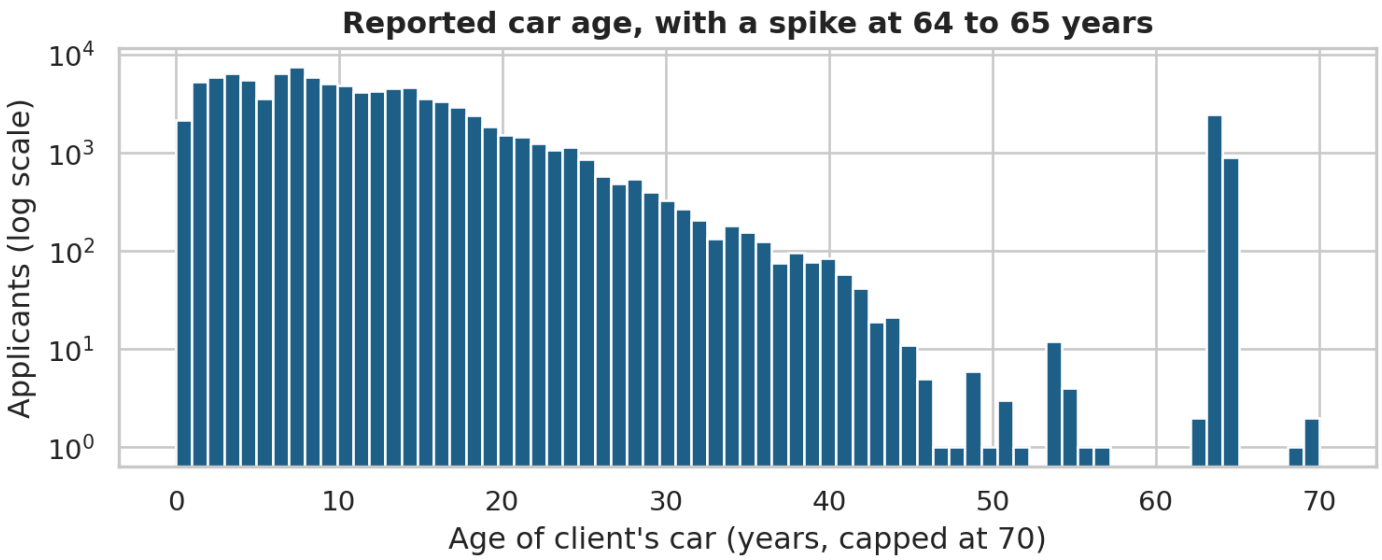
3.3 Which values are impossible or implausible?

```
# Values that contradict the dictionary or that an underwriter would refuse to believe
# Each row: the condition tested, how many applications meet it, and why it is suspect
checks = pd.DataFrame([
    ["DAYS_EMPLOYED = 365,243", (train.DAYS_EMPLOYED == 365243).sum(), "Placeholder;
        99.96% are pensioners"],
    ["OWN_CAR_AGE of 64 or 65 years", train.OWN_CAR_AGE.isin([64, 65]).sum(), "Spike of
        3,334 cars; median owner age 37"],
    ["AMT_INCOME_TOTAL = 117,000,000", (train.AMT_INCOME_TOTAL == 1.17e8).sum(), "Income
        500x the median on a routine loan; client defaulted"],
    ["AMT_REQ_CREDIT_BUREAU_QRT = 261", (train.AMT_REQ_CREDIT_BUREAU_QRT == 261).sum(),
        "261 bureau inquiries in one quarter"],
    ["OBS_30_CNT_SOCIAL_CIRCLE = 348", (train.OBS_30_CNT_SOCIAL_CIRCLE == 348).sum(),
        "Next highest value is 47"],
    ["DAYS_LAST_PHONE_CHANGE = 0", (train.DAYS_LAST_PHONE_CHANGE == 0).sum(), "12% of
        applicants changed phone on application day"],
    ["CODE_GENDER = XNA", (train.CODE_GENDER == "XNA").sum(), "Undocumented level;
        absent from test"],
    ["NAME_FAMILY_STATUS = Unknown", (train.NAME_FAMILY_STATUS == "Unknown").sum(),
        "Undocumented level; absent from test"],
    ["CNT_CHILDREN of 10 or more", (train.CNT_CHILDREN >= 10).sum(), "Up to 19 children;
        possible but rare"],
    ["FLAG_MOBIL = 0", (train.FLAG_MOBIL == 0).sum(), "Column is constant for all but
        one applicant"],
], columns=["Check", "Rows affected", "Why it is suspect"])
checks.style.format({"Rows affected": "{:,}"}).hide(axis="index")
```

Check	Rows affected	Why it is suspect
DAYS_EMPLOYED = 365,243	55,374	Placeholder; 99.96% are pensioners
OWN_CAR_AGE of 64 or 65 years	3,334	Spike of 3,334 cars; median owner age 37
AMT_INCOME_TOTAL = 117,000,000	1	Income 500x the median on a routine loan; client defaulted

Check	Rows affected	Why it is suspect
AMT_REQ_CREDIT_BUREAU_QRT = 261	1	261 bureau inquiries in one quarter
OBS_30_CNT_SOCIAL_CIRCLE = 348	1	Next highest value is 47
DAYS_LAST_PHONE_CHANGE = 0	37,672	12% of applicants changed phone on application day
CODE_GENDER = XNA	4	Undocumented level; absent from test
NAME_FAMILY_STATUS = Unknown	2	Undocumented level; absent from test
CNT_CHILDREN of 10 or more	10	Up to 19 children; possible but rare
FLAG_MOBIL = 0	1	Column is constant for all but one applicant

```
# The car-age spike: a natural age distribution should decline smoothly, not jump at 64-65
fig, ax = plt.subplots(figsize=(8, 3.4))
train["OWN_CAR_AGE"].dropna().clip(upper=70).plot.hist(bins=71, ax=ax, color=PRIMARY)
        # one bin per year
ax.set_yscale("log")                                    # log scale keeps the rare old-car counts visible
ax.set_title("Reported car age, with a spike at 64 to 65 years", fontweight="bold")
ax.set_xlabel("Age of client's car (years, capped at 70)")
ax.set_ylabel("Applicants (log scale)")
plt.tight_layout(); plt.show()
```



- **Finding:** the well-known `DAYS_EMPLOYED` placeholder is only the start. Car age has a second placeholder: 3,334 cars reported as 64 or 65 years old, owned by applicants with a median age of 37, an implausible pattern that reads as a default value in the source system. The 12% of phone changes recorded on application day are likely the same kind of artifact. A single 117 million income is an entry error.
- **Decision:** recode `DAYS_EMPLOYED = 365243` and car ages of 64 to 65 as missing with an indicator flag; cap counts (inquiries, social circle, children) at the 99.9th percentile; drop the 117 million income row from training; drop `FLAG_MOBIL`; fold `XNA` and `Unknown` into missing.

3.4 Are the keys unique and the grain consistent?

```
# Confirm one row per application and no duplicated applications under different IDs
# Duplicate rows after dropping the ID would reveal the same application filed twice
        under new IDs
dup_rows = train.drop(columns=["SK_ID_CURR", "TARGET"]).duplicated().sum()
overlap = len(set(train.SK_ID_CURR) & set(test.SK_ID_CURR))
print(f"Duplicate SK_ID_CURR in train: {train.SK_ID_CURR.duplicated().sum()}")
print(f"Duplicate SK_ID_CURR in test:  {test.SK_ID_CURR.duplicated().sum()}")
print(f"IDs present in both files:     {overlap}")
print(f"Identical rows under different IDs (train): {dup_rows}")
```

```
Duplicate SK_ID_CURR in train: 0
Duplicate SK_ID_CURR in test:  0
IDs present in both files:     0
Identical rows under different IDs (train): 0
```

- **Finding:** the grain holds. Each application appears exactly once, train and test share no applicants, and no application is duplicated under a second ID.
- **Decision:** `SK_ID_CURR` is safe as the join key for the supplementary tables. Because those tables have many rows per applicant, every join must aggregate to one row per `SK_ID_CURR` first, and row counts will be checked after each join.

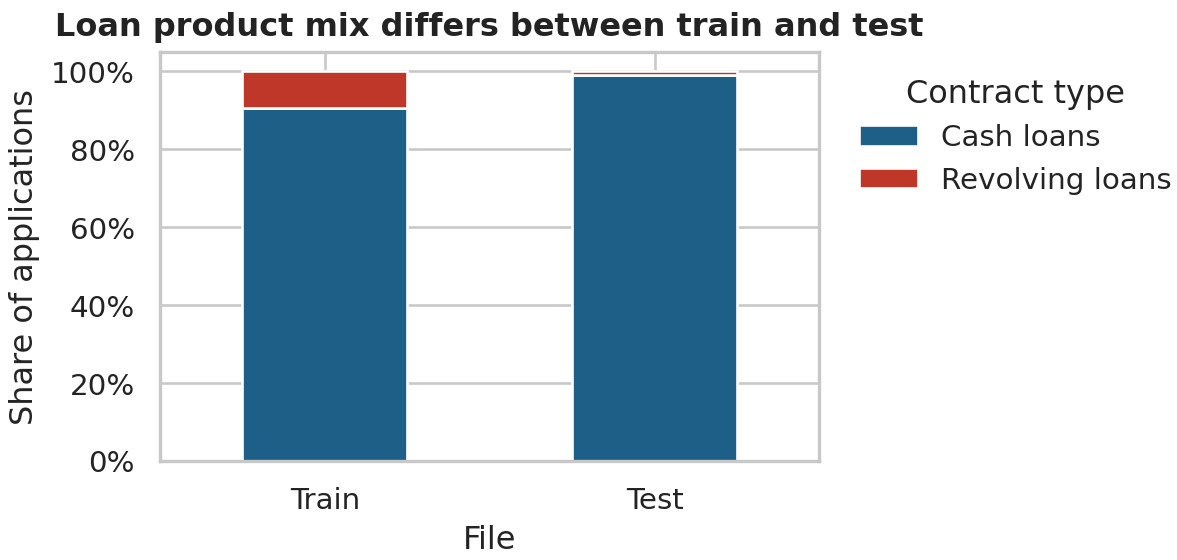
3.5 Do the train and test files describe the same population?

```
# Compare category levels and product mix between the two files
cat_cols = train.select_dtypes(exclude="number").columns
level_gaps = []
for col in cat_cols:
    # Levels that appear in one file but not the other would break or bias a trained
        model
    only_train = set(train[col].dropna()) - set(test[col].dropna())
    only_test = set(test[col].dropna()) - set(train[col].dropna())
    if only_train or only_test:
        level_gaps.append([col, ", ".join(map(str, only_train)) or "none", ",
        ".join(map(str, only_test)) or "none"])
pd.DataFrame(level_gaps, columns=["Column", "Levels only in train", "Levels only in
        test"]).style.hide(axis="index")
```

Column	Levels only in train	Levels only in test
CODE_GENDER	XNA	none
NAME_INCOME_TYPE	Maternity leave	none
NAME_FAMILY_STATUS	Unknown	none

```
# Product mix: share of revolving loans in each file
mix = pd.DataFrame({"Train": train.NAME_CONTRACT_TYPE.value_counts(normalize=True),
                    "Test": test.NAME_CONTRACT_TYPE.value_counts(normalize=True)})
fig, ax = plt.subplots(figsize=(6.5, 3.2))
# One stacked bar per file shows the share of cash vs revolving loans
mix.T.plot.bar(stacked=True, ax=ax, color=[PRIMARY, ACCENT], rot=0)
ax.yaxis.set_major_formatter(mtick.PercentFormatter(1.0, decimals=0))
ax.set_title("Loan product mix differs between train and test", fontweight="bold")
ax.set_xlabel("File")
ax.set_ylabel("Share of applications")
ax.legend(title="Contract type", bbox_to_anchor=(1.02, 1), loc="upper left",
          frameon=False)
plt.tight_layout(); plt.show()

# Document flags that never occur in the test file cannot contribute to scoring it
doc_cols = [c for c in train.columns if c.startswith("FLAG_DOCUMENT")]
never_in_test = [c for c in doc_cols if test[c].sum() == 0]
print(f"{len(never_in_test)} of {len(doc_cols)} document flags are never set in test:
        {', '.join(c.replace('FLAG_DOCUMENT_', '#') for c in never_in_test)}")
```




```
11 of 20 document flags are never set in test: #2, #10, #12, #13, #14, #15, #16, #17, #19,
#20, #21
```

- **Finding:** all 121 shared columns match by name and type, but the populations differ. Revolving loans are 9.5% of training applications and only 0.9% of test applications. Revolving loans default less (5.5% against 8.4% for cash loans), so a model trained on the full mix will be calibrated to a population the test set does not represent. Eleven of the twenty document flags never appear in test, three category levels exist only in training, and one region rating value (-1) appears only in test.
- **Decision:** include contract type as a feature and check performance separately for cash loans, which are 99% of the scoring population. Drop the eleven document flags that never occur in test. Map train-only levels to missing so the preprocessing pipeline cannot fail on unseen categories.

3.6 Was every value known at the time of application?

```
# Every DAYS_ column counts days before the application, so valid values are zero or
        negative.
# Report the latest value in each column, excluding the known 365,243 employment
        placeholder.
days_cols = [c for c in train.columns if c.startswith("DAYS_")]
latest = pd.DataFrame({
    "Latest value in train": [train[c].where(train[c] != 365243).max() for c in
        days_cols],
    "Latest value in test": [test[c].where(test[c] != 365243).max() for c in days_cols],
}, index=days_cols)
latest.style.format("{:,.0f}")
```

	Latest value in train	Latest value in test
DAYS_BIRTH	-7,489	-7,338
DAYS_EMPLOYED	0	-1
DAYS_REGISTRATION	0	0
DAYS_ID_PUBLISH	0	0
DAYS_LAST_PHONE_CHANGE	0	0

- **Test:** every `DAYS_*` column should be zero or negative, meaning it was known on or before the application date.
- **Finding:** all `DAYS_*` values are at or before the application date except the `DAYS_EMPLOYED` placeholder handled above. The application tables therefore contain only decision-time information.
- **Decision:** no leakage risk at this stage. When bureau and previous-application tables are joined, any record dated after the application will be excluded before aggregation.

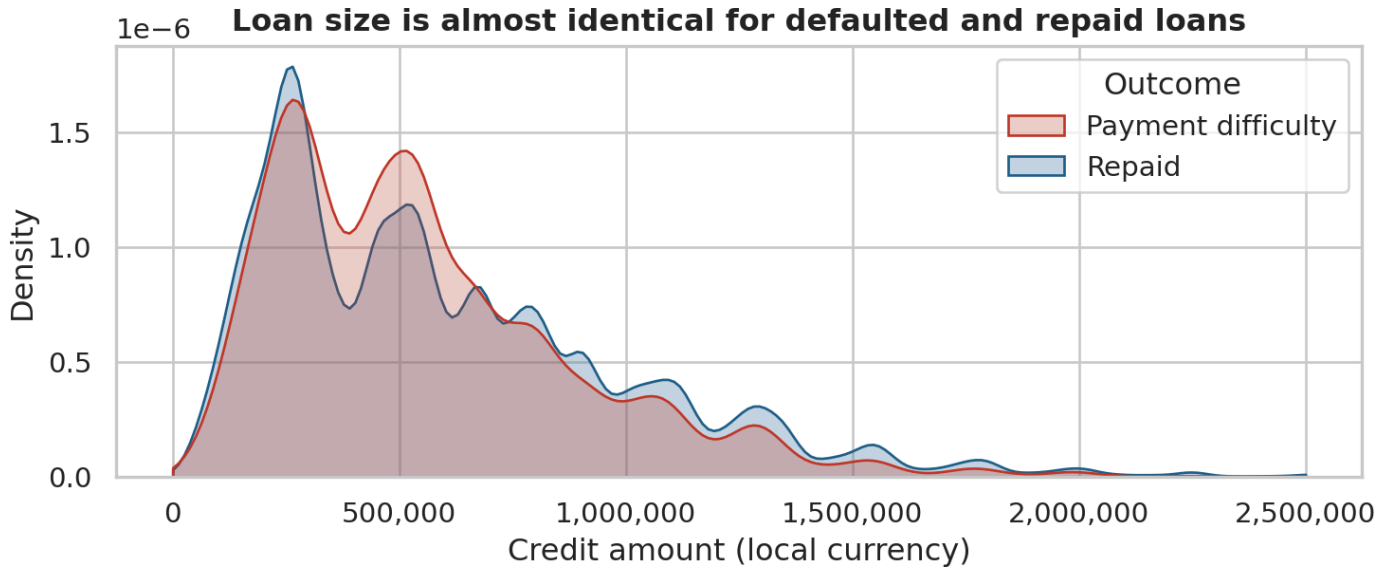
4 Exploratory results

4.1 Q1. What does a wrong decision cost the lender, and can this data measure it?

```
# Exposure: how much credit sits in loans that ended in payment difficulty?
exposure = train.groupby("TARGET")["AMT_CREDIT"].agg(loans="size", total_credit="sum",
        median_credit="median")
# Share of all credit issued that sits in each outcome group
exposure["share_of_credit"] = exposure.total_credit / exposure.total_credit.sum()
exposure.index = ["Repaid", "Payment difficulty"]

# Compare the shape of the loan-size distribution for the two outcomes (each curve
        scaled to its own group)
fig, ax = plt.subplots(figsize=(8, 3.6))
```

```
sns.kdeplot(data=train.assign(Outcome=train.TARGET.map(OUTCOME)), x="AMT_CREDIT",
        hue="Outcome",
            common_norm=False, clip=(0, 2.5e6), palette=OUTCOME_COLORS, ax=ax,
        fill=True, alpha=0.25)
ax.set_title("Loan size is almost identical for defaulted and repaid loans",
        fontweight="bold")
ax.set_xlabel("Credit amount (local currency)")
ax.set_ylabel("Density")
ax.xaxis.set_major_formatter(mtick.StrMethodFormatter("{x:,.0f}"))
plt.tight_layout(); plt.show()
exposure.style.format({"loans": "{:,}", "total_credit": "{:,.0f}", "median_credit": "
        {:,.0f}", "share_of_credit": "{:.1%}"})
```



	loans	total_credit	median_credit	share_of_credit
Repaid	282,686	170,360,232,246	517,788	92.5%
Payment difficulty	24,825	13,846,851,950	497,520	7.5%

Interpretation. Loans that ended in payment difficulty hold 7.5% of all credit issued, about 13.8 billion in local currency, and their median size (497,520) is nearly the same as repaid loans (517,788). For the lender, that means loan size does not flag risk: defaults are spread across ordinary loans rather than concentrated in a few large ones. The data can measure exposure but not the full cost of a wrong decision, because it contains neither the loss given default (how much is recovered) nor the margin earned on a good loan. Those two numbers set the price of a false approval relative to a false decline.

Decision: evaluate the model with a cost-weighted threshold analysis using a range of assumed cost ratios (for example, one false approval costing five, ten, or twenty times one false decline), and flag loss given default and loan margin as inputs to request from the business.

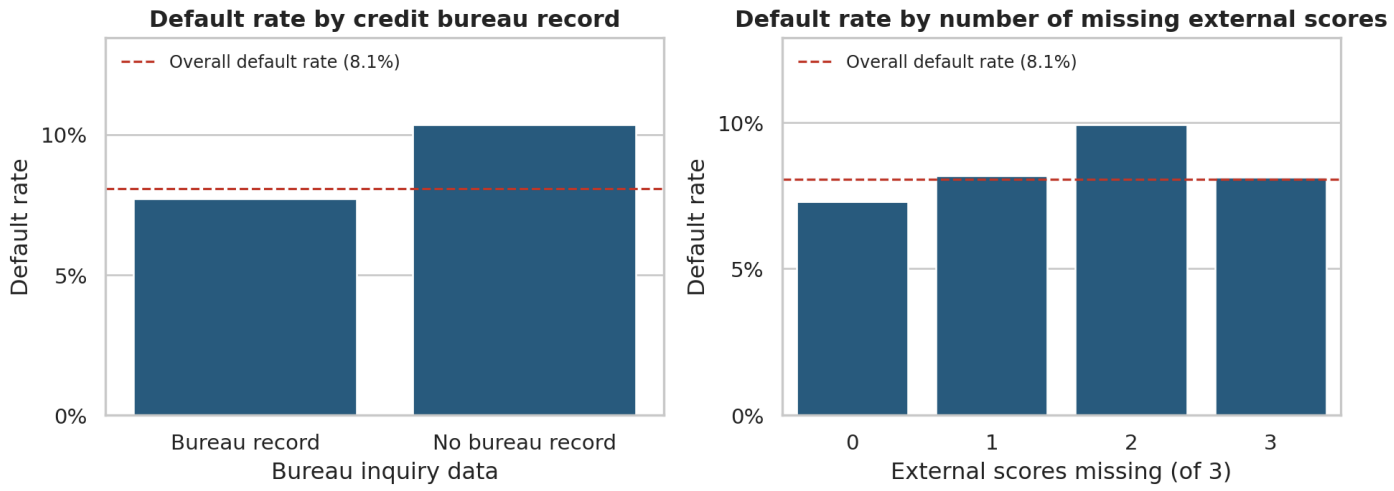
4.2 Q2. What share of applicants are thin-file, and do they default differently?

```
# Thin-file proxy: no credit bureau inquiry record at all, and how many external scores
        are missing
no_bureau = train["AMT_REQ_CREDIT_BUREAU_YEAR"].isna()
n_ext_missing = train[["EXT_SOURCE_1", "EXT_SOURCE_2",
        "EXT_SOURCE_3"]].isna().sum(axis=1)

# Default rate for applicants with and without a bureau record, and by count of missing
        scores
thin = rate_by_group(train.assign(bureau=np.where(no_bureau, "No bureau record", "Bureau
        record")), "bureau")
ext_miss = rate_by_group(train.assign(ext_missing=n_ext_missing.astype(str)),
        "ext_missing")

fig, axes = plt.subplots(1, 2, figsize=(10, 3.8))
plot_rate(thin, "bureau", "Default rate by credit bureau record", "Bureau inquiry data",
        ax=axes[0])
plot_rate(ext_miss, "ext_missing", "Default rate by number of missing external scores",
        "External scores missing (of 3)", ax=axes[1])
plt.tight_layout(); plt.show()

print(f"No bureau record: {no_bureau.mean():.1%} of applicants. "
      f"Of these, {train.loc[no_bureau, 'EXT_SOURCE_3'].isna().mean():.0%} are also
        missing EXT_SOURCE_3.")
```




```
No bureau record: 13.5% of applicants. Of these, 100% are also missing EXT_SOURCE_3.
```

Interpretation. 13.5% of applicants have no credit bureau inquiry record, and they default at 10.3% against 7.7% for applicants with one, a gap of more than a third. Every one of those applicants is also missing `EXT_SOURCE_3`, which ties the two gaps to a single cause: no bureau file exists for them. Default risk climbs with each missing external score, from 7.3% with all three present to 9.9% with two missing. This confirms the BPS premise. Thin-file applicants are a large, identifiable, and riskier segment, and they are the segment where a better model has the most room to help and the most room to exclude creditworthy people.

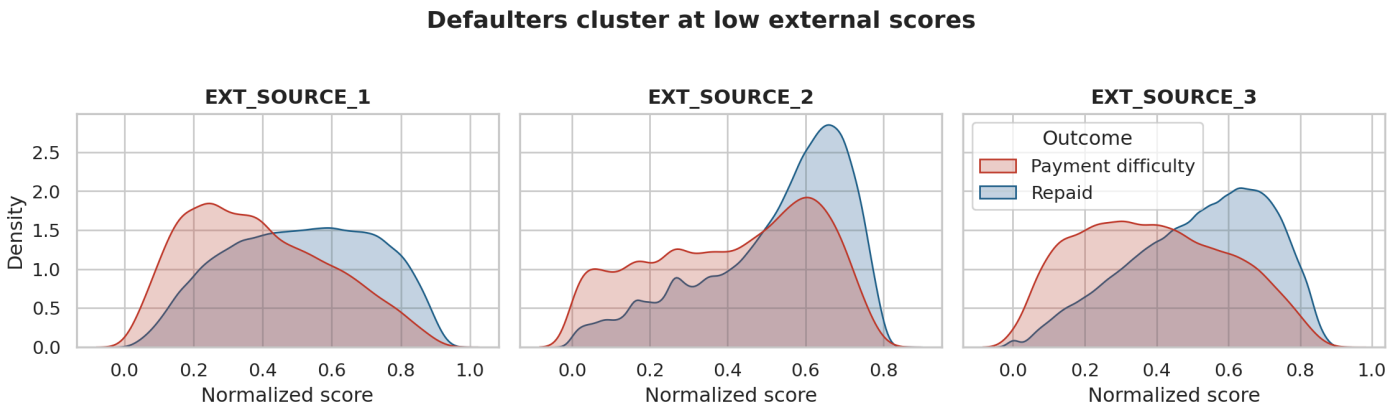
Decision: create a `thin_file` flag (no bureau record) and a count of missing external scores, keep thin-file applicants in the same model, and report model performance for the thin-file segment separately so the approval-rate target in the BPS can be tracked where it matters. Missing inquiry

counts are a proxy, not proof, of a missing bureau file, so the flag will be confirmed against `bureau.csv` coverage when that table is joined.

4.3 Q3. How well do the external scores separate defaulters, and does combining them help?

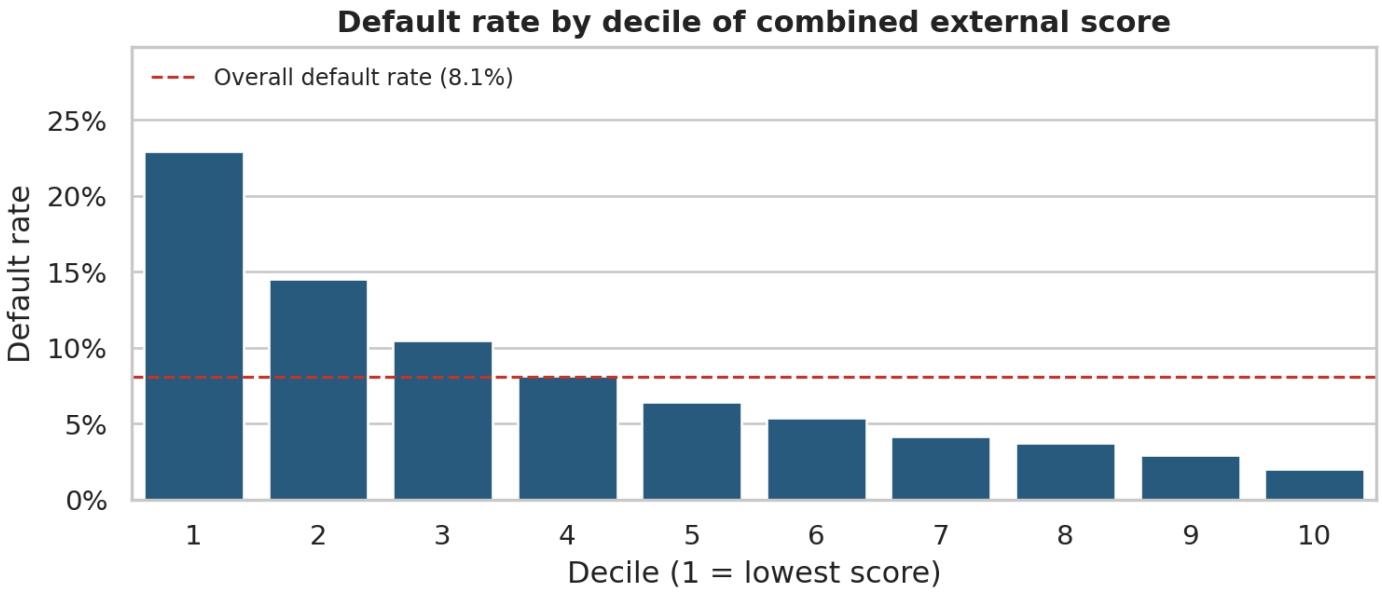
```
# Distribution of each external score by outcome, and the signal of each alone and
        combined
ext_cols = ["EXT_SOURCE_1", "EXT_SOURCE_2", "EXT_SOURCE_3"]
fig, axes = plt.subplots(1, 3, figsize=(12, 3.4), sharey=True)
for ax, col in zip(axes, ext_cols):                          # one density panel per score
    sns.kdeplot(data=train.assign(Outcome=train.TARGET.map(OUTCOME)), x=col,
        hue="Outcome", common_norm=False,
                palette=OUTCOME_COLORS, fill=True, alpha=0.25, ax=ax, legend=(col ==
        ext_cols[-1]))
    ax.set_title(col, fontweight="bold"); ax.set_xlabel("Normalized score");
        ax.set_ylabel("Density")
fig.suptitle("Defaulters cluster at low external scores", fontweight="bold", y=1.03)
plt.tight_layout(); plt.show()

# AUC of each score alone and of the average of whichever scores an applicant has
ext_signal = pd.DataFrame(
    [(c, *auc_signal(train[c]), train[c].notna().mean()) for c in ext_cols] +
    [("Mean of available scores", *auc_signal(train[ext_cols].mean(axis=1)),
        train[ext_cols].notna().any(axis=1).mean())],
    columns=["Score", "AUC", "Direction", "Coverage"])
ext_signal.style.format({"AUC": "{:.3f}", "Coverage": "{:.1%}"}).hide(axis="index")
```



Score	AUC	Direction	Coverage
EXT_SOURCE_1	0.666	-	43.6%
EXT_SOURCE_2	0.656	-	99.8%
EXT_SOURCE_3	0.679	-	80.2%
Mean of available scores	0.717	-	99.9%

```
# Default rate across deciles of the combined external score
ext_mean = train[ext_cols].mean(axis=1)                    # mean ignores missing scores
# Split applicants into ten equal-size groups by combined score
deciles = rate_by_group(train.assign(decile=pd.qcut(ext_mean, 10, labels=range(1, 11))),
        "decile")
fig, ax = plt.subplots(figsize=(8, 3.6))
plot_rate(deciles, "decile", "Default rate by decile of combined external score",
        "Decile (1 = lowest score)", ax=ax)
plt.tight_layout(); plt.show()
```



Interpretation. The three external scores are the strongest single signals in the data, each reaching an AUC between 0.66 and 0.68, and they are only weakly correlated with each other, so each carries information the others do not. Averaging whatever scores an applicant has lifts the AUC to about 0.72 and covers nearly every applicant. The decile chart shows why a lender should care: default runs from 23% in the lowest decile to 2% in the highest, an eleven-fold spread. The mechanism is that these scores summarize repayment behavior observed elsewhere, which is exactly what a thin-file lender lacks internally. Their weakness is coverage: `EXT_SOURCE_1` is missing for 56% of applicants.

Decision: keep all three scores, add the mean and the count of available scores as engineered features, and impute missing scores inside the modeling pipeline rather than dropping them. Because these scores come from outside vendors, request their definitions before the model card stage, since an unexplained vendor score is hard to defend to a declined applicant.

4.4 Q4. Is reported income believable, and is payment-to-income predictive?

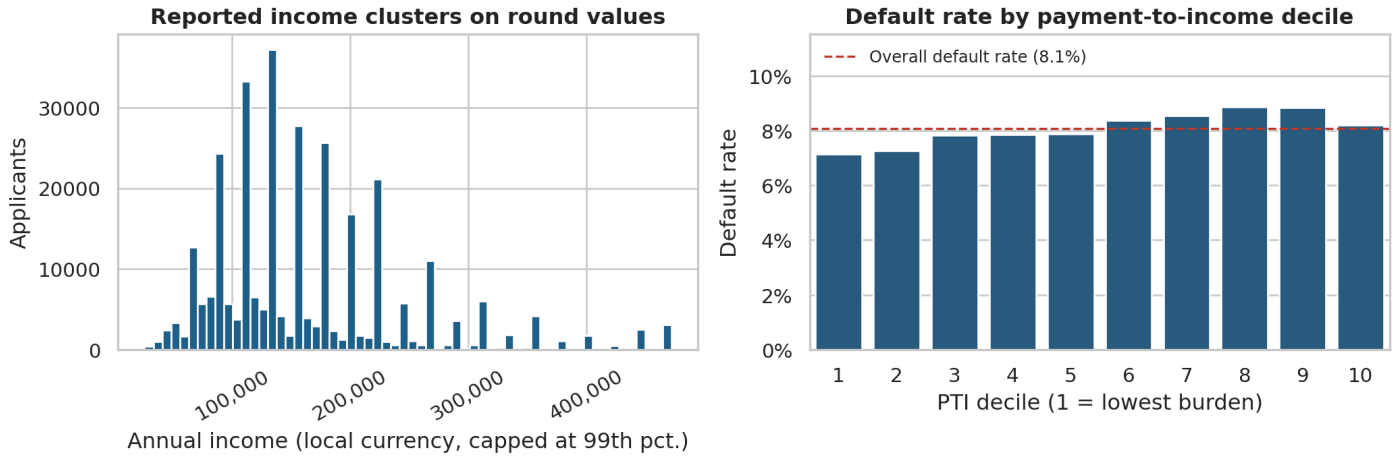
```
# Income heaping: how concentrated are reported incomes on a few round values?
income = train["AMT_INCOME_TOTAL"]
top10_share = income.value_counts().head(10).sum() / len(income)
print(f"Top 10 income values cover {top10_share:.0%} of applicants; "
```

```
      f"{(income % 4500 == 0).mean():.1%} of incomes are exact multiples of 4,500.")

# Payment-to-income: annual annuity divided by annual income
pti = train["AMT_ANNUITY"] / income
# Default rate within each tenth of the PTI distribution
pti_tbl = rate_by_group(train.assign(pti_decile=pd.qcut(pti, 10, labels=range(1, 11))),
        "pti_decile")

fig, axes = plt.subplots(1, 2, figsize=(11, 3.8))
# Cap at the 99th percentile so the handful of extreme incomes do not flatten the
        histogram
income.clip(upper=income.quantile(0.99)).plot.hist(bins=60, ax=axes[0], color=PRIMARY)
axes[0].set_title("Reported income clusters on round values", fontweight="bold")
axes[0].set_xlabel("Annual income (local currency, capped at 99th pct.)")
axes[0].set_ylabel("Applicants")
axes[0].xaxis.set_major_formatter(mtick.StrMethodFormatter("{x:,.0f}"))
axes[0].tick_params(axis="x", rotation=30)
plot_rate(pti_tbl, "pti_decile", "Default rate by payment-to-income decile", "PTI decile
        (1 = lowest burden)", ax=axes[1])
plt.tight_layout(); plt.show()
print(f"AUC: income alone {auc_signal(income)[0]:.3f}, payment-to-income
        {auc_signal(pti)[0]:.3f}")
```

```
Top 10 income values cover 67% of applicants; 97.5% of incomes are exact multiples of
4,500.
```




```
AUC: income alone 0.519, payment-to-income 0.520
```

Interpretation. Reported income does not look verified. Two thirds of applicants report one of just ten values, and 97.5% of incomes are exact multiples of 4,500, the pattern of a self-reported, rounded figure. That matters because payment-to-income, the central ability-to-repay measure in U.S. regulation, is only as good as the income behind it. Here PTI carries weak signal (AUC 0.52): default rises from 7.1% in the lowest-burden decile to 8.9% in the eighth and then dips. The mechanism is likely noise in the denominator rather than an absence of relationship; an applicant who overstates income appears less burdened than they are.

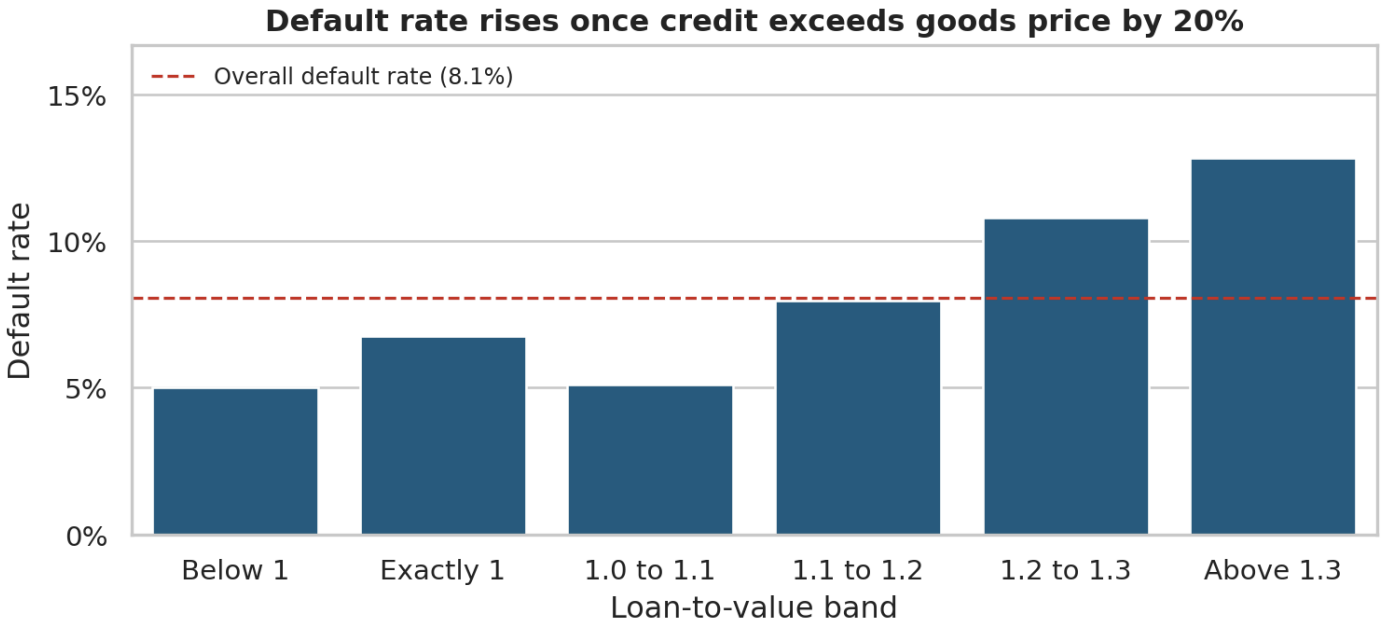
Decision: build PTI and keep it as a candidate feature because regulators and underwriters expect it and it is easy to explain, but do not rely on it. Flag income verification as a data gap for the BPS: if

Home Credit holds verified income (for example, from payroll or bank records), PTI should be rebuilt on it.

4.5 Q5. How often does the loan exceed the goods price, and is loan-to-value predictive?

```
# Loan-to-value: credit amount relative to the price of the goods being financed
ltv = train["AMT_CREDIT"] / train["AMT_GOODS_PRICE"]
# Bands isolate loans financed exactly at the goods price from those above and below it
ltv_band = pd.cut(ltv, [0, 0.9999, 1.0001, 1.1, 1.2, 1.3, np.inf],
                  labels=["Below 1", "Exactly 1", "1.0 to 1.1", "1.1 to 1.2", "1.2 to
        1.3", "Above 1.3"])
ltv_tbl = rate_by_group(train.assign(ltv_band=ltv_band), "ltv_band")

fig, ax = plt.subplots(figsize=(8, 3.8))
plot_rate(ltv_tbl, "ltv_band", "Default rate rises once credit exceeds goods price by
        20%", "Loan-to-value band", ax=ax)
plt.tight_layout(); plt.show()
print(f"LTV above 1: {(ltv > 1.0001).mean():.1%} of loans. Median LTV
        {ltv.median():.3f}. AUC {auc_signal(ltv)[0]:.3f}")
ltv_tbl.style.format({"default_rate": "{:.2%}", "applicants": "
        {:,}"}).hide(axis="index")
```




```
LTV above 1: 64.6% of loans. Median LTV 1.119. AUC 0.569
```

ltv_band	default_rate	applicants
Below 1	5.00%	260
Exactly 1	6.75%	108,210

ltv_band	default_rate	applicants
1.0 to 1.1	5.11%	27,161
1.1 to 1.2	7.97%	106,919
1.2 to 1.3	10.80%	35,159
Above 1.3	12.82%	29,524

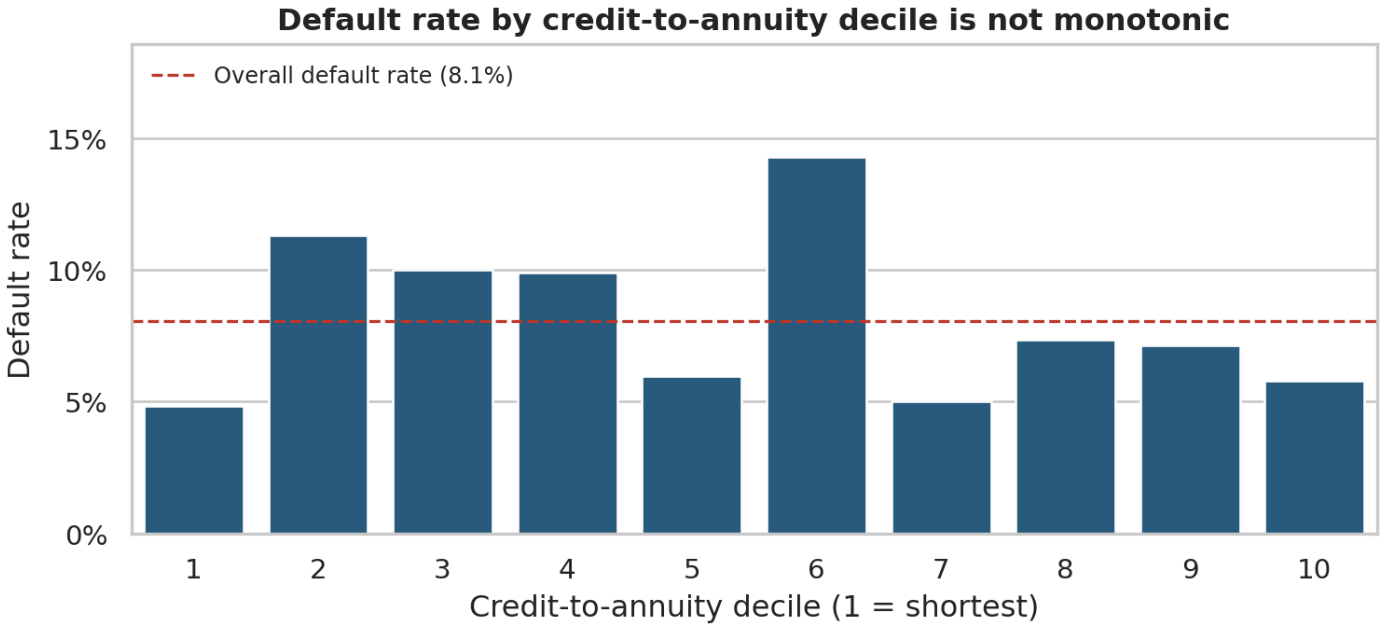
Interpretation. Almost two thirds of loans are larger than the price of the goods they finance, with a median loan-to-value of 1.12. The likely mechanism is that insurance, fees, or cash add-ons are rolled into the loan. The pattern is not linear: loans financed at or just above the goods price default at 5.1% to 6.8%, while loans more than 30% above it default at 12.8%, about twice the rate. For the lender, a large financed add-on means the customer is borrowing more than the purchase requires, which is a behavioral risk signal and also the part of the loan with nothing behind it.

Decision: build LTV as a feature (AUC 0.57, the strongest of the ratio features) and keep its non-linear shape by letting the model split it rather than forcing a straight line. The 278 revolving loans with no goods price get LTV set to missing with an indicator.

4.6 Q6. Does the ratio of credit to annuity (implied term) relate to default?

```
# Credit divided by annuity approximates the number of payments, but it also absorbs the
        interest rate
term = train["AMT_CREDIT"] / train["AMT_ANNUITY"]
# Default rate within each tenth of the credit-to-annuity distribution
term_tbl = rate_by_group(train.assign(term_decile=pd.qcut(term, 10, labels=range(1,
        11))), "term_decile")

fig, ax = plt.subplots(figsize=(8, 3.8))
plot_rate(term_tbl, "term_decile", "Default rate by credit-to-annuity decile is not
        monotonic", "Credit-to-annuity decile (1 = shortest)", ax=ax)
plt.tight_layout(); plt.show()
print(f"Credit-to-annuity ranges {term.min():.1f} to {term.max():.1f}. AUC
        {auc_signal(term)[0]:.3f}")
```




```
Credit-to-annuity ranges 8.0 to 45.3. AUC 0.532
```

Interpretation. The implied term does not rise or fall smoothly with risk; default jumps between 4.8% and 14.3% from one decile to the next. That zigzag is the finding. Credit divided by annuity is not a pure loan term: for a fixed number of payments, a higher interest rate raises the annuity and lowers the ratio. The ratio therefore blends the term with the price Home Credit charged, and the price was set using Home Credit's own view of the applicant's risk. The deciles are most likely picking up discrete pricing tiers, a hypothesis the contracted term in the previous applications table can test.

Decision: keep the credit-to-annuity ratio as a feature, because it carries the lender's prior risk assessment, and use a tree-based model that can learn its step pattern. Document it in the model card as an indirect use of past pricing decisions, and request the actual contracted term (`CNT_PAYMENT` in the previous applications table) to separate term from price.

4.7 Q7. Does employment tenure predict default once the placeholder value is handled?

```
# Replace the 365,243 placeholder with missing and convert days to years of tenure
tenure = -train["DAYS_EMPLOYED"].where(train["DAYS_EMPLOYED"] != 365243) / 365.25
tenure_band = pd.cut(tenure, [-0.01, 1, 2, 5, 10, 20, 50],
                     labels=["Under 1", "1 to 2", "2 to 5", "5 to 10", "10 to 20",
        "20+"])
# Keep the placeholder rows visible as their own band instead of silently dropping them
tenure_band = tenure_band.cat.add_categories("Placeholder
        (pensioner)").fillna("Placeholder (pensioner)")
tenure_tbl = rate_by_group(train.assign(tenure_band=tenure_band), "tenure_band")

fig, ax = plt.subplots(figsize=(9, 3.8))
plot_rate(tenure_tbl, "tenure_band", "Default rate falls steadily with years in current
        job", "Years in current employment", rotate=15, ax=ax)
```

```
plt.tight_layout(); plt.show()
print(f"AUC of tenure after removing the placeholder: {auc_signal(tenure)[0]:.3f}")
```




```
AUC of tenure after removing the placeholder: 0.582
```

Interpretation. Once the placeholder is removed, tenure is one of the cleaner relationships in the data: default runs about 11% for applicants in their first two years of a job and falls to 4.2% for those with twenty or more years. The mechanism is income stability, which is why employment status is one of the CFPB's ability-to-repay factors. The placeholder rows are not noise either. They are almost all pensioners, who default at only 5.4%, and replacing their value with a typical tenure would erase a genuinely low-risk group.

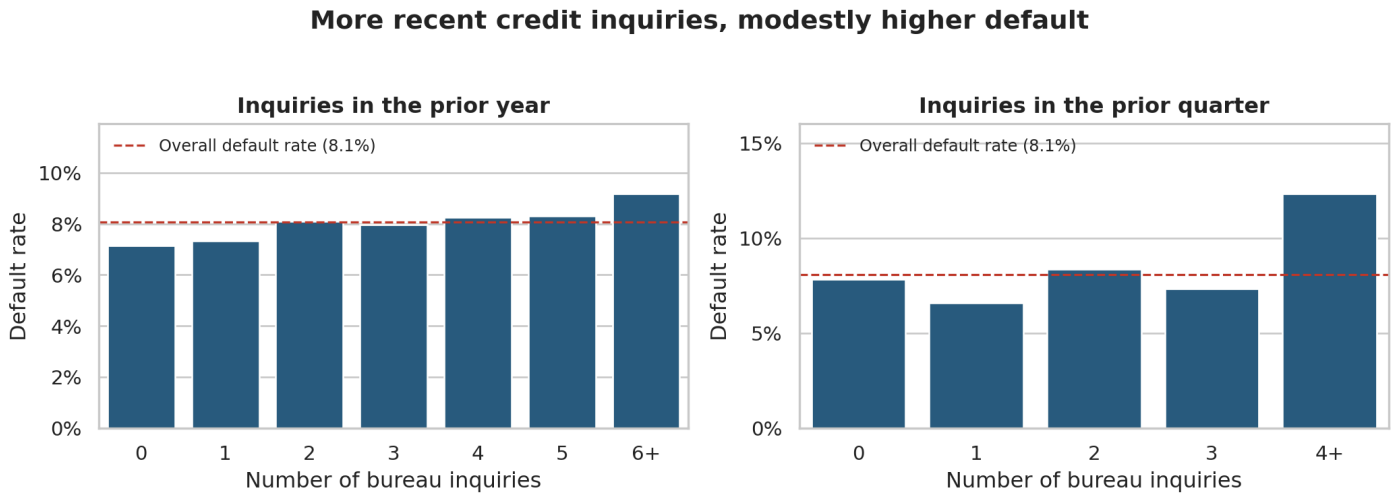
Decision: recode the placeholder as missing, add an `is_pensioner` flag, and convert tenure to years. Keep the raw placeholder out of every numeric calculation, since it would distort any mean or scaling.

4.8 Q8. Does recent credit-seeking behavior signal risk?

```
# Bureau inquiries in the prior year (excluding the last quarter) and the prior quarter
# Cap counts so the long tail forms one "6+" or "4+" group, then label as text for
        plotting
year_q =
        train["AMT_REQ_CREDIT_BUREAU_YEAR"].clip(upper=6).astype("Int64").astype(str).re
        place("6", "6+")
qrt_q =
        train["AMT_REQ_CREDIT_BUREAU_QRT"].clip(upper=4).astype("Int64").astype(str).rep
        lace("4", "4+")
year_tbl =
        rate_by_group(train.loc[train.AMT_REQ_CREDIT_BUREAU_YEAR.notna()].assign(n=year_
        q), "n")
qrt_tbl =
        rate_by_group(train.loc[train.AMT_REQ_CREDIT_BUREAU_QRT.notna()].assign(n=qrt_q)
        , "n")
# Sort the text labels numerically (0, 1, 2, ... 6+) rather than alphabetically
```

```
order = lambda t: t.assign(k=t.n.str.replace("+", "",
        regex=False).astype(int)).sort_values("k")

fig, axes = plt.subplots(1, 2, figsize=(11, 3.8))
plot_rate(order(year_tbl), "n", "Inquiries in the prior year", "Number of bureau
        inquiries", ax=axes[0])
plot_rate(order(qrt_tbl), "n", "Inquiries in the prior quarter", "Number of bureau
        inquiries", ax=axes[1])
fig.suptitle("More recent credit inquiries, modestly higher default", fontweight="bold",
        y=1.03)
plt.tight_layout(); plt.show()
print(f"AUC: yearly inquiries {auc_signal(train.AMT_REQ_CREDIT_BUREAU_YEAR)[0]:.3f}, "
      f"quarterly {auc_signal(train.AMT_REQ_CREDIT_BUREAU_QRT)[0]:.3f}")
```




```
AUC: yearly inquiries 0.521, quarterly 0.506
```

Interpretation. The relationship exists but is weak. Default rises from 7.1% with no yearly inquiries to 9.2% with six or more, and the 584 applicants with four or more inquiries in the prior quarter default at 12.3%. The mechanism the World Bank describes holds: an applicant shopping for credit in several places may be under financial strain. But inquiry counts add little on their own (AUC 0.52 or lower). The larger signal in this group is the one already found in Q2: having no inquiry record at all.

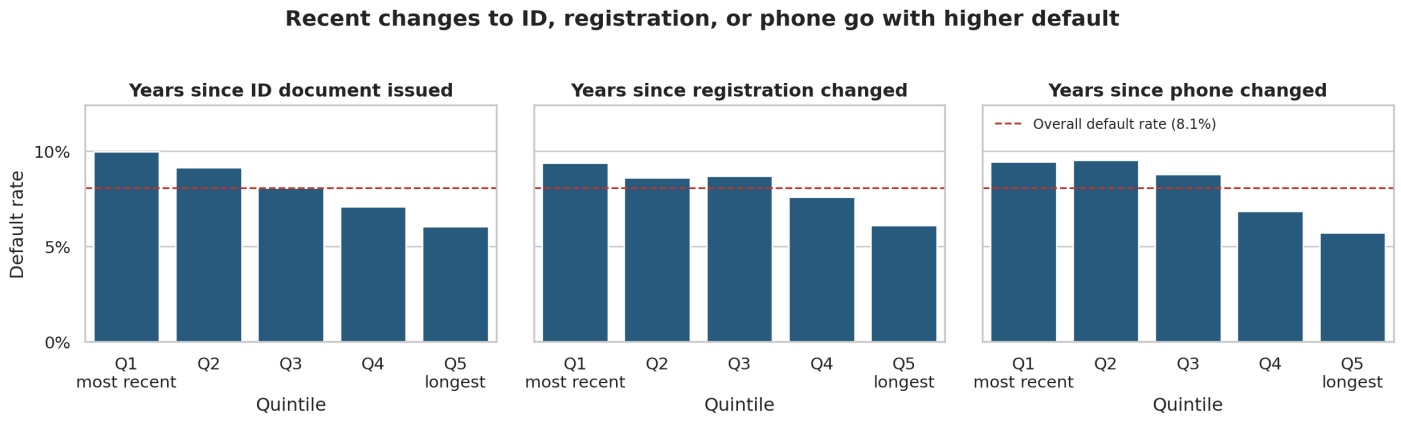
Decision: keep a single total-inquiries feature (capped) and the thin-file flag from Q2; drop the hour, day, and week inquiry columns, which are almost always zero.

4.9 Q9. Do residential and identity stability indicators relate to default?

```
# Convert days-before-application to years, then compare default rates across quintiles
stability = {"DAYS_ID_PUBLISH": "Years since ID document issued",
             "DAYS_REGISTRATION": "Years since registration changed",
             "DAYS_LAST_PHONE_CHANGE": "Years since phone changed"}
fig, axes = plt.subplots(1, 3, figsize=(13, 3.8), sharey=True)
for ax, (col, label) in zip(axes, stability.items()):
    yrs = -train[col] / 365.25                          # negative days before
        application -> positive years
```

```
    # Quintile 1 holds the most recent changes, quintile 5 the longest-standing
    tbl = rate_by_group(train.assign(q=pd.qcut(yrs, 5, labels=["Q1\nmost recent", "Q2",
        "Q3", "Q4", "Q5\nlongest"])), "q")
    plot_rate(tbl, "q", label, "Quintile", ax=ax)
    if ax is not axes[-1]:                              # keep one legend for the three
        panels
        ax.get_legend().remove()
fig.suptitle("Recent changes to ID, registration, or phone go with higher default",
        fontweight="bold", y=1.03)
plt.tight_layout(); plt.show()

# Signal of each indicator on its own
signal = pd.DataFrame([(stability[c], *auc_signal(train[c])) for c in stability],
        columns=["Indicator", "AUC", "Direction"])
signal.style.format({"AUC": "{:.3f}"}).hide(axis="index")
```



Indicator	AUC	Direction
Years since ID document issued	0.556	+
Years since registration changed	0.543	+
Years since phone changed	0.557	+

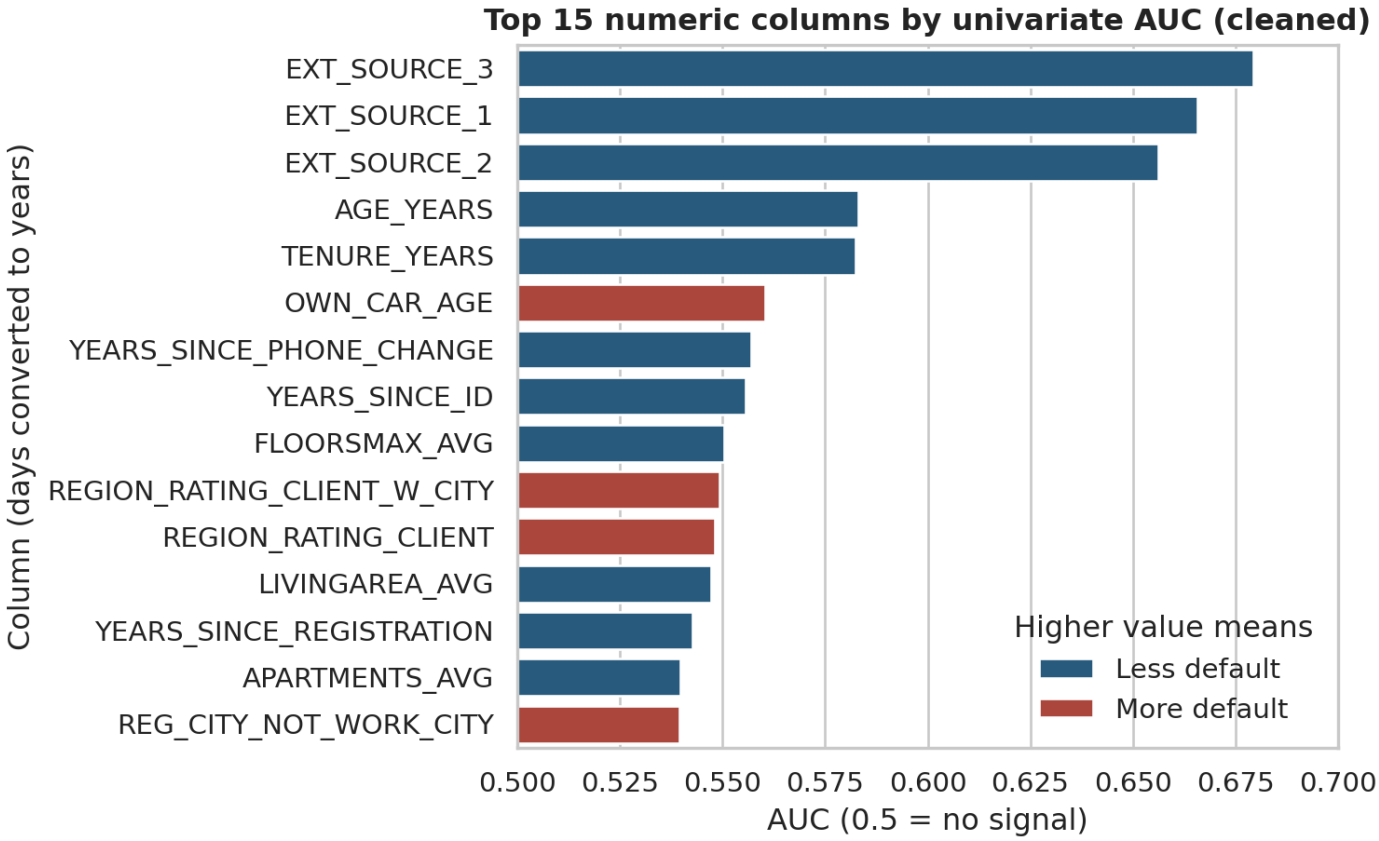
Interpretation. All three indicators point the same way: applicants who recently changed their ID document, registration, or phone default more often, about 9.4% to 10% in the most recent quintile against about 6% in the longest. The mechanism is life stability. People who have recently moved, replaced documents, or switched phones are more often in transition, and these columns are the closest the application data comes to the “length at address” measure lenders use. Each alone is modest (AUC about 0.54 to 0.56), but they are cheap, decision-time, and easy to explain. One caution: phone change is recorded as day zero for 12% of applicants, and that group defaults at 9.7%, so the zero behaves partly as its own category.

Decision: keep all three as years-since features and add a flag for phone change recorded on the application day, so the model can separate the artifact from genuine recency.

4.10 Q10. Which applicant characteristics carry the most signal overall?

```
# Rank on cleaned columns so directions read naturally and placeholders do not distort
        the result:
# days become positive years, placeholders become missing, and the building block keeps
        only _AVG columns
numeric = train.select_dtypes("number").drop(columns=["SK_ID_CURR", "TARGET"]).copy()
numeric = numeric.drop(columns=[c for c in numeric if c.endswith(("_MODE", "_MEDI"))])
numeric["DAYS_EMPLOYED"] = numeric["DAYS_EMPLOYED"].where(numeric["DAYS_EMPLOYED"] !=
        365243)
numeric["OWN_CAR_AGE"] = numeric["OWN_CAR_AGE"].where(~numeric["OWN_CAR_AGE"].isin([64,
        65]))
days_cols = [c for c in numeric if c.startswith("DAYS_")]
numeric[days_cols] = -numeric[days_cols] / 365.25
numeric = numeric.rename(columns={"DAYS_BIRTH": "AGE_YEARS", "DAYS_EMPLOYED":
        "TENURE_YEARS",
                                  "DAYS_ID_PUBLISH": "YEARS_SINCE_ID",
        "DAYS_REGISTRATION": "YEARS_SINCE_REGISTRATION",
                                  "DAYS_LAST_PHONE_CHANGE": "YEARS_SINCE_PHONE_CHANGE"})
# Univariate AUC and coverage for every column that is not constant; keep the top 15
auc_rows = [(c, *auc_signal(numeric[c]), numeric[c].notna().mean())
            for c in numeric.columns if numeric[c].nunique() > 1]
auc_tbl = (pd.DataFrame(auc_rows, columns=["column", "auc", "direction", "coverage"])
           .sort_values("auc", ascending=False).head(15))
auc_tbl["Higher value means"] = auc_tbl.direction.map({"+": "More default", "-": "Less
        default"})

fig, ax = plt.subplots(figsize=(8, 5))
sns.barplot(data=auc_tbl, y="column", x="auc", hue="Higher value means", dodge=False,
            palette={"More default": ACCENT, "Less default": PRIMARY}, ax=ax)
ax.axvline(0.5, color=GREY, lw=1)
ax.set_xlim(0.5, 0.7)
ax.set_title("Top 15 numeric columns by univariate AUC (cleaned)", fontweight="bold")
ax.set_xlabel("AUC (0.5 = no signal)")
ax.set_ylabel("Column (days converted to years)")
ax.legend(title="Higher value means", frameon=False, loc="lower right")
plt.tight_layout(); plt.show()
```




```
# Categorical columns: spread between the riskiest and safest level (levels with 500+
        applicants)
cat_rows = []
for col in ["OCCUPATION_TYPE", "NAME_EDUCATION_TYPE", "NAME_HOUSING_TYPE",
        "NAME_INCOME_TYPE",
            "REGION_RATING_CLIENT", "NAME_CONTRACT_TYPE", "FLAG_OWN_CAR"]:
    # Treat every level as text (so numeric ratings group cleanly) and ignore levels too
        small to trust
    g = rate_by_group(train.assign(**{col: train[col].astype(str)}), col); g =
        g[g.applicants >= 500]
    lo, hi = g.loc[g.default_rate.idxmin()], g.loc[g.default_rate.idxmax()]
    cat_rows.append([col, lo[col], lo.default_rate, hi[col], hi.default_rate])
cat_tbl = pd.DataFrame(cat_rows, columns=["Column", "Safest level", "Default rate
        (safest)", "Riskiest level", "Default rate (riskiest)"])
cat_tbl.style.format({"Default rate (safest)": "{:.1%}", "Default rate (riskiest)": "
        {:.1%}"}).hide(axis="index")
```

Column	Safest level	Default rate (safest)	Riskiest level	Default rate (riskiest)
OCCUPATION_TYPE	Accountants	4.8%	Low-skill Laborers	17.2%
NAME_EDUCATION_TYPE	Higher education	5.4%	Lower secondary	10.9%
NAME_HOUSING_TYPE	Office apartment	6.6%	Rented apartment	12.3%
NAME_INCOME_TYPE	Pensioner	5.4%	Working	9.6%
REGION_RATING_CLIENT	1	4.8%	3	11.1%

Column	Safest level	Default rate (safest)	Riskiest level	Default rate (riskiest)
NAME_CONTRACT_TYPE	Revolving loans	5.5%	Cash loans	8.3%
FLAG_OWN_CAR	Y	7.2%	N	8.5%

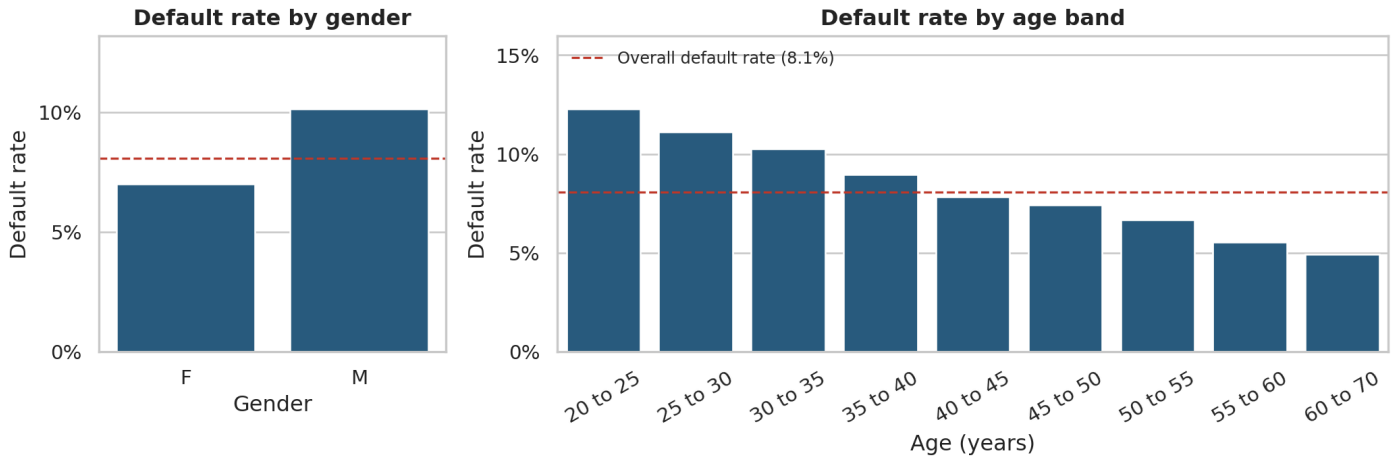
Interpretation. The ranking sorts the data into three tiers. The external scores stand alone at the top (AUC 0.66 to 0.68). A second tier between roughly 0.54 and 0.58 holds age, employment tenure, car age, the stability indicators, and the regional rating, with a few building statistics at the bottom of that range. Everything else falls below 0.54. Among categories, occupation and education show the widest spreads: low-skill laborers default at 17.2% against 4.8% for accountants, and lower-secondary education at 10.9% against 5.4% for higher education. No single variable is strong, which is typical in credit scoring and is why lenders combine many modest signals.

Decision: carry the external scores, age, tenure, stability indicators, regional rating, LTV, credit-to-annuity ratio, occupation, education, and housing type into modeling as the core shortlist, and test the building statistics as a reduced set rather than all 47.

4.11 Q11. How strongly do protected attributes relate to default, and which predictors act as proxies for them?

```
# Default rate by gender and by age band
age = -train["DAYS_BIRTH"] / 365.25
age_edges = [20, 25, 30, 35, 40, 45, 50, 55, 60, 70]
age_labels = [f"{lo} to {hi}" for lo, hi in zip(age_edges[:-1], age_edges[1:])]
age_tbl = rate_by_group(train.assign(age_band=pd.cut(age, age_edges,
        labels=age_labels)), "age_band")
gender_tbl = rate_by_group(train[train.CODE_GENDER != "XNA"], "CODE_GENDER")   # drop
        the 4 undocumented rows

fig, axes = plt.subplots(1, 2, figsize=(11, 3.8), gridspec_kw={"width_ratios": [1,
        2.4]})
plot_rate(gender_tbl, "CODE_GENDER", "Default rate by gender", "Gender", ax=axes[0])
plot_rate(age_tbl, "age_band", "Default rate by age band", "Age (years)", rotate=30,
        ax=axes[1])
axes[0].get_legend().remove()
plt.tight_layout(); plt.show()
```



```
# Proxy check: which predictors are strongly associated with age or gender?
candidates = {"EXT_SOURCE_1": train.EXT_SOURCE_1, "EXT_SOURCE_2": train.EXT_SOURCE_2,
              "EXT_SOURCE_3": train.EXT_SOURCE_3, "Tenure in years": tenure}
female, male = train.CODE_GENDER == "F", train.CODE_GENDER == "M"
# A strong correlation with age, or a large female/male gap, marks a column as a
        possible proxy
proxy = pd.DataFrame({name: {"Correlation with age": age.corr(x),
                             "Mean if female": x[female].mean(),
                             "Mean if male": x[male].mean()}
                      for name, x in candidates.items()}).T
proxy.style.format("{:.2f}")
```

	Correlation with age	Mean if female	Mean if male
EXT_SOURCE_1	0.60	0.55	0.41
EXT_SOURCE_2	0.09	0.52	0.51
EXT_SOURCE_3	0.21	0.51	0.50
Tenure in years	0.35	6.99	5.77

Interpretation. Both attributes are strongly associated with default. Men default at 10.1% against 7.0% for women, and default falls steadily with age, from 12.3% for applicants in their early twenties to 4.9% for those over sixty. Neither can be used freely. Under U.S. Regulation B, sex may not be considered at all, and age may be used in a scoring system only if older applicants are not penalized; the EU AI Act classifies credit scoring as a high-risk system. The more important finding is the proxy check: `EXT_SOURCE_1` correlates 0.60 with age and averages 0.55 for women against 0.41 for men. The strongest predictor in the data partly encodes the protected attributes, so dropping `CODE_GENDER` alone would not remove gender or age from the model. Occupation is similar, since drivers and low-skill laborers are 85% to 95% male.

Decision: exclude `CODE_GENDER` from the model. Keep age, which is permitted and runs in the direction the regulation allows (older applicants score as lower risk). Keep `EXT_SOURCE_1` and occupation, but test the final model for disparate impact by gender and age band before any recommendation, and document both proxies in the model card.

5 Results

5.1 What data problems did the EDA discover, and what will be done about each?

Problem	Evidence	Decision for data preparation
Employment placeholder	55,374 rows at +365,243 days, almost all pensioners	Recode to missing; add `is_pensioner` flag

Problem	Evidence	Decision for data preparation
Car age placeholder	3,334 cars reported as 64 to 65 years old	Recode to missing; add flag
Informative missingness	No building data: 9.3% default vs 7.0%; no bureau record: 10.3% vs 7.7%	Add missing indicators before imputing; do not drop rows
Redundant building block	47 columns, three statistics of the same measures, missing together	Keep `_AVG` versions and one completeness flag
Entry errors and extreme counts	Income of 117 million; 261 quarterly inquiries; social circle of 348	Drop the income row; cap counts at the 99.9th percentile
Unverified income	97.5% of incomes are multiples of 4,500	Use PTI with caution; request verified income
Train/test product mix shift	Revolving loans 9.5% of train, 0.9% of test	Keep contract type; validate on cash loans separately
Features absent from test	11 of 20 document flags never set in test; `FLAG_MOBIL` constant	Drop them
Undocumented and train-only levels	`XNA` gender, `Unknown` family status, `Maternity leave`, `-1` region rating	Map to missing
Phone change on application day	12% of applicants, default 9.7%	Add a same-day flag

5.2 Which relationships with default are particularly strong?

1. **External scores.** Each reaches an AUC of 0.66 to 0.68; their mean reaches 0.72, and default ranges from 23% in the lowest decile to 2% in the highest.

2. **Employment tenure.** Default runs about 11% in the first two years of a job and falls to 4.2% after twenty years.
3. **Age.** Default falls from 12.3% in the early twenties to 4.9% over sixty.
4. **Loan-to-value.** Loans more than 30% above the goods price default at 12.8%, about twice the rate of loans at the goods price.
5. **Stability indicators.** Recent ID, registration, or phone changes go with default near 10% against 6% for the most stable applicants.
6. **Occupation and education.** Spreads of 4.8% to 17.2% and 5.4% to 10.9%.

5.3 What did the EDA confirm or change in the business problem statement?

- **Confirmed:** thin-file applicants are a large (13.5%) and riskier segment, so the BPS was right to target them. No single variable is strong, which supports a model that combines many modest signals.
- **Changed:** the BPS success metrics assume we can value a false approval against a false decline, but this data holds neither loss given default nor loan margin. The threshold analysis will need assumed cost ratios, and the BPS should list these as inputs to obtain.
- **Changed:** the BPS called fairness a constraint to check at the end. The EDA shows the strongest predictor is itself correlated with age and gender, so fairness testing must be built into model selection, not added afterward.
- **Changed:** income, the basis of the regulator's payment-to-income test, appears to be self-reported and rounded. Verified income should be added to the data requests.
- **Added:** the product mix differs between training and scoring populations, so model performance must be reported for cash loans specifically.

5.4 How has the EDA influenced the analytics approach?

A gradient-boosted tree model is the natural next step. It handles missing values and non-linear shapes (LTV, credit-to-annuity) without forcing them into straight lines, and the EDA shows several of the most useful signals are neither linear nor complete. A logistic regression will be kept as an interpretable benchmark, because every decision must be explainable to a declined applicant.

6 Use of AI: how I used it and how it went

I used Claude (Anthropic) as an analyst, following the stage-gated workflow from class:

1. **Learn the data.** Claude profiled both application files and the dictionary: grain, key uniqueness, train/test overlap, and columns whose values contradict their definitions.
2. **Research industry features.** Before writing questions, Claude gathered sources on how lenders assess repayment risk (CFPB ability-to-repay factors, Regulation B, the World Bank credit scoring guidelines, the 2019 interagency statement on alternative data, the EU AI Act, and peer-reviewed work) and mapped each feature to the columns that could build it. That became `docs/feature_report.md`.
3. **Plan and execute.** I set the question list from my BPS and the feature report; Claude checked it against the standing integrity checks and wrote and ran the code one question at a time.
4. **Interpret and decide.** Claude drafted interpretations and proposed data preparation decisions. I reviewed each one against the output, edited or rejected what did not hold up, and made the

final calls. I then asked what had been missed against the BPS, the dictionary, and what an underwriter would refuse to believe. That pass surfaced the car-age placeholder, the product-mix shift, the document flags absent from test, and the age and gender correlation in `EXT_SOURCE_1`.

How it went. The AI was fastest and most useful for profiling and for code: checks that would have taken an evening took minutes. Its weakest moments were first drafts that described output rather than explaining it, and a tendency to reach for generic profiling (a correlation heatmap, a missingness bar chart) instead of questions tied to the lender's decision. The quality came from forcing every section to end in a decision and from checking every number in the text against the output.

7 References

- Consumer Financial Protection Bureau. 12 CFR 1026.43(c)(2), Ability-to-Repay underwriting factors.
- Consumer Financial Protection Bureau. 12 CFR 1002.6 (Regulation B), rules concerning evaluation of applications.
- Consumer Financial Protection Bureau, Office of Research (2015). *Data Point: Credit Invisibles*.
- Federal Reserve, CFPB, FDIC, NCUA, and OCC (2019). *Interagency Statement on the Use of Alternative Data in Credit Underwriting*.
- World Bank and International Committee on Credit Reporting (2019). *Credit Scoring Approaches Guidelines*.
- European Union (2024). Artificial Intelligence Act, Annex III, point 5(b).
- Berg, T., Burg, V., Gombović, A., and Puri, M. (2020). On the rise of FinTechs: Credit scoring using digital footprints. *Review of Financial Studies*, 33(7), 2845 to 2897.
- Kuhn, M., and Johnson, K. (2013). *Applied Predictive Modeling*, Chapter 3: Data Pre-Processing. Springer.
- Home Credit Group. Home Credit Default Risk competition data and column descriptions, Kaggle.